



Universidade de São Paulo – USP

**Programa de Educação Continuada da Escola Politécnica da USP
– PECE**

Curso de Especialização em Engenharia de Dados e Big Data

Disciplina: Ecossistemas de Big Data

Relatório Técnico da Implementação

Estudo de caso: Execução do WordCount MapReduce no AWS EMR com Terraform

Ambiente: *AWS Academy Learner Lab* | Execução local: *Windows 11/WSL 2* | Execução
remota: *Amazon EMR*

Grupo A

Nome	Matrícula USP
Breno Tostes Garcia	18105650
Gerson Chadi Junior	
Isabella Abreu Comelli	
Fernando Luiz Gomes	
Caetano Sales Carvalho	

São Paulo, 17 de julho de 2026

SUMÁRIO

1	Introdução.....	3
2	Objetivos	4
3	Método De Execução E Registro	4
4	Ambiente Computacional.....	5
5	Arquitetura Implementada.....	5
6	Preparação E Validação Do Ambiente	6
7	Revisão Técnica E Ajustes No <i>Terraform</i>	7
8	Provisionamento Da Infraestrutura	8
9	Execução Do Processamento <i>MapReduce</i>	9
10	Resultados E Análise.....	9
11	Problemas Encontrados E Soluções Adotadas	12
12	Evidências, Rastreabilidade E Reprodutibilidade	12
13	Descomissionamento E Controle De Custos.....	13
14	Conclusão	14
15	Apêndice A – Sequência Essencial De Comandos	15

RESUMO EXECUTIVO

Este relatório registra a preparação, correção, execução e validação de uma aplicação *WordCount MapReduce* em um *cluster Amazon EMR* provisionado por *Terraform* no *AWS Academy Learner Lab*. A atividade foi conduzida em um computador com *Windows 11* e *WSL 2*, utilizando *AWS CLI 2.36.1*, *Terraform 1.15.8* e *OpenJDK 11.0.31*. A infraestrutura criada compreendeu um *bucket Amazon S3* e um *cluster EMR 6.15.0* formado por um nó master e um nó core, ambos do tipo *m4.large*. O fluxo operacional contemplou a disponibilização dos códigos *Java* e do arquivo de entrada no *S3*, a compilação do *JAR* no *cluster*, a transferência da entrada para o *HDFS*, a execução do processamento *MapReduce* e a cópia do resultado para o *S3*. Os quatro *Steps* do *EMR* foram concluídos com sucesso, gerando os arquivos `_SUCCESS` e `part-r-00000`. O resultado contabilizou 1.651 ocorrências distribuídas em 1.045 tokens distintos, segundo uma tokenização sensível a maiúsculas, minúsculas e pontuação. As evidências foram preservadas localmente, incluindo outputs do *Terraform*, descrição do *cluster*, estados dos *Steps*, fontes *Java*, *JAR*, logs e arquivos de saída. No último registro disponível, o *cluster* encontrava-se em estado *WAITING*, de modo que o descomissionamento ainda deveria ser confirmado para interromper a geração de custos.

Palavras-chave: *Amazon EMR; Hadoop; MapReduce; WordCount; Terraform; Amazon S3; WSL 2.*

1 INTRODUÇÃO

O processamento distribuído de dados é um componente central das arquiteturas contemporâneas de engenharia de dados, sobretudo quando o volume de informações ultrapassa a capacidade prática de uma única máquina. Nesse contexto, o ecossistema *Hadoop* oferece mecanismos para armazenamento distribuído e execução paralela, enquanto o *Amazon EMR* disponibiliza esses componentes como serviço gerenciado na nuvem. O *Terraform*, por sua vez, permite descrever e reproduzir a infraestrutura por meio de código, reduzindo a dependência de configurações manuais no console da *AWS*.

A atividade teve como objeto a execução do algoritmo *WordCount*, um exemplo clássico de *MapReduce* que transforma um arquivo textual em pares compostos por *token* e

frequência. Embora simples, o exercício permite observar o ciclo completo de uma carga distribuída: preparação do ambiente local, provisionamento de recursos, transferência de dados entre *S3* e *HDFS*, compilação e execução de código *Java*, monitoramento de *Steps*, persistência do resultado e organização das evidências.

O relatório enfatiza não apenas o resultado final, mas também as decisões técnicas e as correções necessárias para tornar o tutorial executável no *Windows 11* por meio do *WSL 2*. Essa abordagem favorece a reprodutibilidade do experimento e explicita as diferenças entre comandos executados localmente e comandos destinados ao ambiente do *cluster EMR*.

2 OBJETIVOS

O objetivo geral foi provisionar e validar uma arquitetura *Hadoop* gerenciada na *AWS*, executando um *WordCount MapReduce* com infraestrutura definida em *Terraform* e armazenamento persistente no *Amazon S3*.

Os objetivos específicos foram:

- validar o ambiente local *Windows 11/WSL 2*, incluindo *AWS CLI*, *Terraform* e *Java*;
- confirmar as credenciais temporárias, as roles do *EMR* e a chave *EC2* do *Learner Lab*;
- revisar o código *Terraform* e corrigir os *Steps* incompatíveis com o fluxo operacional do *EMR*;
- provisionar o *bucket S3*, os objetos de entrada e o *cluster EMR*;
- compilar e executar a aplicação *Java MapReduce* no *cluster*;
- validar a saída do *WordCount* e preservar evidências suficientes para auditoria e reprodução;
- registrar a necessidade de descomissionamento dos recursos após a conclusão.

3 MÉTODO DE EXECUÇÃO E REGISTRO

A execução foi conduzida de forma incremental. Cada etapa foi validada antes do avanço para a seguinte, evitando a criação de recursos com configurações incorretas. O procedimento foi dividido em cinco fases: diagnóstico do ambiente local; inspeção e ajuste dos

arquivos *Terraform*; geração e aplicação do plano; monitoramento do *cluster* e dos *Steps*; e validação e preservação das evidências.

Os registros considerados neste relatório foram produzidos no terminal *Ubuntu* do *WSL* 2 e incluem saídas do *AWS STS*, *Terraform*, *Amazon EMR* e *Amazon S3*. Nenhuma credencial secreta foi incorporada ao documento. Os valores apresentados referem-se aos identificadores operacionais necessários para reprodução e verificação do experimento.

4 AMBIENTE COMPUTACIONAL

Tabela 4-1: Componentes do ambiente utilizado

Componente	Configuração observada
Sistema operacional local	<i>Windows 11</i> com <i>WSL 2</i> e <i>Ubuntu 24.04</i>
Kernel <i>WSL</i>	6.6.87.2-microsoft-standard-WSL2
<i>AWS CLI</i>	2.36.1
<i>Terraform</i>	1.15.8, provider <i>AWS</i> 5.x
<i>Java</i>	OpenJDK/Javac 11.0.31
Ambiente de nuvem	<i>AWS Academy Learner Lab</i>
Conta <i>AWS</i>	650379606721
Região	us-east-1
<i>Amazon EMR</i>	Release 6.15.0, aplicação <i>Hadoop</i>
Nós do <i>cluster</i>	1 master <i>m4.large</i> e 1 core <i>m4.large</i>
Armazenamento persistente	<i>Amazon S3</i>
Chave <i>EC2</i>	vockey; arquivo privado labsuser.pem

5 ARQUITETURA IMPLEMENTADA

A arquitetura combinou o *S3* como repositório persistente e o *HDFS* como sistema de arquivos temporário do *cluster*. O computador local foi responsável por autenticar a sessão, validar o *Terraform* e iniciar o provisionamento. Os arquivos `lorem.txt`, fontes *Java* e

`bootstrap.sh` foram armazenados no *S3*. No *EMR*, o código foi compilado em `WordCount.JAR`, o arquivo de entrada foi transferido para `HDFS:///input/`, o job `MapReduce` produziu `HDFS:///output/WordCount/` e o resultado foi copiado novamente para o *S3*.

Arquitetura operacional do WordCount no AWS EMR

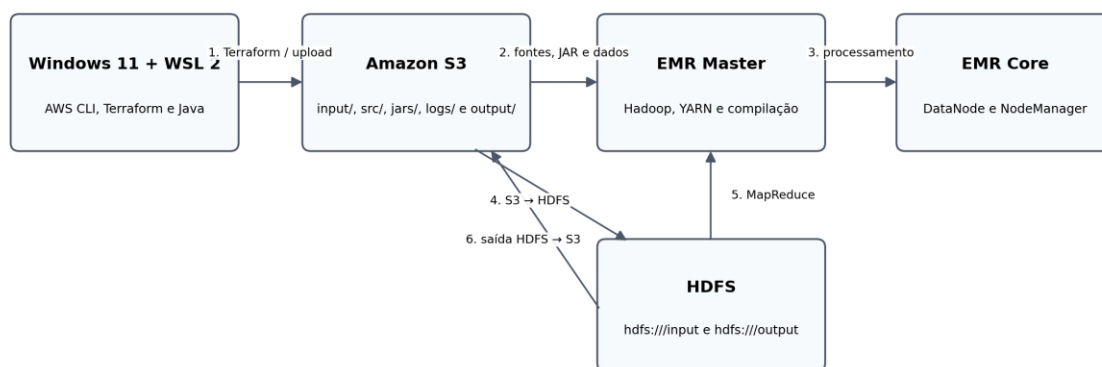


Figura 5-1: Arquitetura operacional do processamento *WordCount*

Tabela 5-1: Recursos previstos no plano do *Terraform*

Recurso	Finalidade	Quantidade
<code>AWS_S3_bucket.EMR_lab</code>	<code>Bucket 650379606721-EMR-lab-WordCount</code>	1
<code>AWS_S3_object.input_data</code>	Arquivo <code>input/lorem.txt</code>	1
<code>AWS_S3_object.Java_sources</code>	Três arquivos <i>Java</i> em <code>src/</code>	3
<code>AWS_S3_object.bootstrap_script</code>	Script <code>scripts/bootstrap.sh</code>	1
<code>AWS_EMR_cluster.WordCount</code>	<i>Cluster EMR</i> e quatro <i>Steps</i>	1

6 PREPARAÇÃO E VALIDAÇÃO DO AMBIENTE

A validação inicial confirmou a disponibilidade das ferramentas necessárias e a validade das credenciais temporárias. O comando `AWS sts get-caller-identity` identificou a conta `650379606721` e a `role voclabs`. Também foram confirmadas as `roles EMR_DefaultRole` e `EMR_EC2_DefaultRole`, bem como a `key pair vockey`. Não havia *cluster EMR* ativo nem estado *Terraform* anterior na pasta de trabalho.

```
AWS -version
Terraform version
Java -version
Javac -version
AWS sts get-caller-identity
AWS iam list-roles --query 'Roles[?RoleName==`EMR_DefaultRole` ||
RoleName==`EMR_EC2_DefaultRole`].RoleName'
AWS EC2 describe-key-pairs --query 'KeyPairs[].KeyName'
```

O diretório do projeto encontrava-se em uma pasta do *OneDrive* montada no *WSL*. Os scripts foram convertidos para o padrão de final de linha do *Linux* e tiveram as permissões de execução revisadas. A opção pelo *WSL 2* permitiu manter o ambiente *Windows 11* sem misturar sintaxes do *PowerShell* e do *Bash*.

7 REVISÃO TÉCNICA E AJUSTES NO *TERRAFORM*

A inspeção do *main.tf* identificou dois padrões operacionais com elevada probabilidade de falha. O primeiro executava *Hadoop JAR* diretamente sobre uma *URI S3://*, embora o comando *Hadoop* utilizado pelo *Step* esperasse um arquivo local. O segundo utilizava *S3-dist-cp* na direção *HDFS* para *S3*, fluxo que havia apresentado comportamento inconsistente no tutorial. Antes do *apply*, os *Steps* 3 e 4 foram corrigidos.

Tabela 7-1: Correções aplicadas nos *Steps* do *Amazon EMR*

Etapa	Configuração original	Ajuste adotado
Step 3 – WordCount	<i>Hadoop JAR</i> <i>S3://.../WordCount.JAR</i>	Baixar o <i>JAR</i> com <i>AWS S3 cp</i> para <i>/tmp/WordCount.JAR</i> e executar <i>Hadoop JAR</i> sobre o caminho local.
Step 4 – saída	<i>S3-dist-cp</i> de <i>HDFS</i> para <i>S3</i>	Executar <i>HDFS dfs -copyToLocal</i> e, em seguida, <i>AWS S3 cp --recursive</i> para o prefixo <i>output/</i> .

```
# Step 3 corrigido
set -e && AWS S3 cp S3://${local.bucket_name}/JARs/WordCount.JAR
/tmp/WordCount.JAR && Hadoop JAR /tmp/WordCount.JAR WordCountApplication
HDFS:///input/ HDFS:///output/WordCount/

# Step 4 corrigido
set -e && rm -rf /tmp/wc-output && mkdir -p /tmp/wc-output && HDFS dfs -
copyToLocal HDFS:///output/WordCount/* /tmp/wc-output/ && AWS S3 cp
/tmp/wc-output/ S3://${local.bucket_name}/output/ --recursive
```

Após as alterações, *Terraform* *fmt* e *Terraform* *validate* foram executados com sucesso. O plano final indicou sete recursos a criar, nenhum recurso a alterar e nenhum recurso a destruir. O plano foi salvo em *Terraform*/*tfplan* para que o *apply* executasse exatamente a configuração revisada.

8 PROVISIONAMENTO DA INFRAESTRUTURA

O provisionamento foi iniciado com `Terraform -chdir=Terraform apply tfplan`. O *bucket S3* foi criado primeiro, seguido pelos objetos de entrada, fontes *Java* e *script* de *bootstrap*. O *cluster EMR* foi provisionado em aproximadamente 3 minutos e 57 segundos. O *Terraform* concluiu o *apply* com sete recursos adicionados e nenhum recurso alterado ou destruído.

```
Terraform -chdir=Terraform plan -out=tfplan
Terraform -chdir=Terraform apply tfplan
```

Tabela 8-1: Identificadores da infraestrutura criada

Elemento	Valor
<i>Cluster EMR</i>	j-MUPMC8FELJBR
Nome do <i>cluster</i>	<i>WordCount-EMR-cluster</i>
DNS do master	<i>EC2-18-208-194-148.compute-1.AmazonAWS.com</i>
<i>Bucket S3</i>	<i>650379606721-EMR-lab-WordCount</i>
Saída <i>S3</i>	<i>S3:///650379606721-EMR-lab-WordCount/output/</i>
Estado após execução	<i>WAITING</i>

9 EXECUÇÃO DO PROCESSAMENTO *MAPREDUCE*

A execução foi organizada em quatro *Steps* sequenciais. O *Step 1* compilou os três arquivos *Java* utilizando o *classpath* do *Hadoop* instalado no *EMR* e publicou o *JAR* no *S3*. O *Step 2* transferiu o arquivo `lorem.txt` do *S3* para o *HDFS*. O *Step 3* executou o *WordCount*, e o *Step 4* copiou a saída do *HDFS* para o prefixo `output/` do *bucket*.

Tabela 9-1: Estados finais dos *Steps* do *cluster*

<i>Step</i>	Finalidade	Estado
<i>Step1-Compile-JAR</i>	Compilar fontes <i>Java</i> e publicar <i>WordCount.JAR</i>	COMPLETED
<i>Step2-Copy-Input-S3-to-HDFS</i>	Transferir <code>input/</code> para <i>HDFS:///input/</i>	COMPLETED
<i>Step3-WordCount-MapReduce</i>	Executar <i>WordCountApplication</i>	COMPLETED
<i>Step4-Copy-Output-HDFS-to-S3</i>	Persistir <code>part-r-00000</code> e <code>_SUCCESS</code> no <i>S3</i>	COMPLETED

A conclusão integral dos quatro *Steps* demonstrou que o fluxo de compilação, movimentação de dados, execução distribuída e persistência da saída funcionou conforme a arquitetura proposta. O arquivo `WordCount.JAR` foi publicado no *S3* e os arquivos de saída foram gerados com uma diferença aproximada de 4 minutos..

10 RESULTADOS E ANÁLISE

A saída foi armazenada nos objetos `output/_SUCCESS` e `output/part-r-00000`. O primeiro, com tamanho zero, sinaliza que a tarefa foi concluída sem erro. O segundo, com 9.256 bytes, contém os pares token–frequência produzidos pelo *reducer*. A leitura do resultado identificou 1.651 ocorrências distribuídas em 1.045 tokens distintos.

Tabela 10-1: Vinte tokens mais frequentes

Posição	Token	Frequência
1	the	60
2	a	47
3	of	38
4	and	30

Posição	Token	Frequência
5	to	25
6	in	22
7	I	18
8	you	13
9	on	12
10	that	12
11	was	11
12	your	11
13	from	10
14	his	10
15	as	9
16	he	9
17	with	9
18	for	8
19	it	8
20	me	6

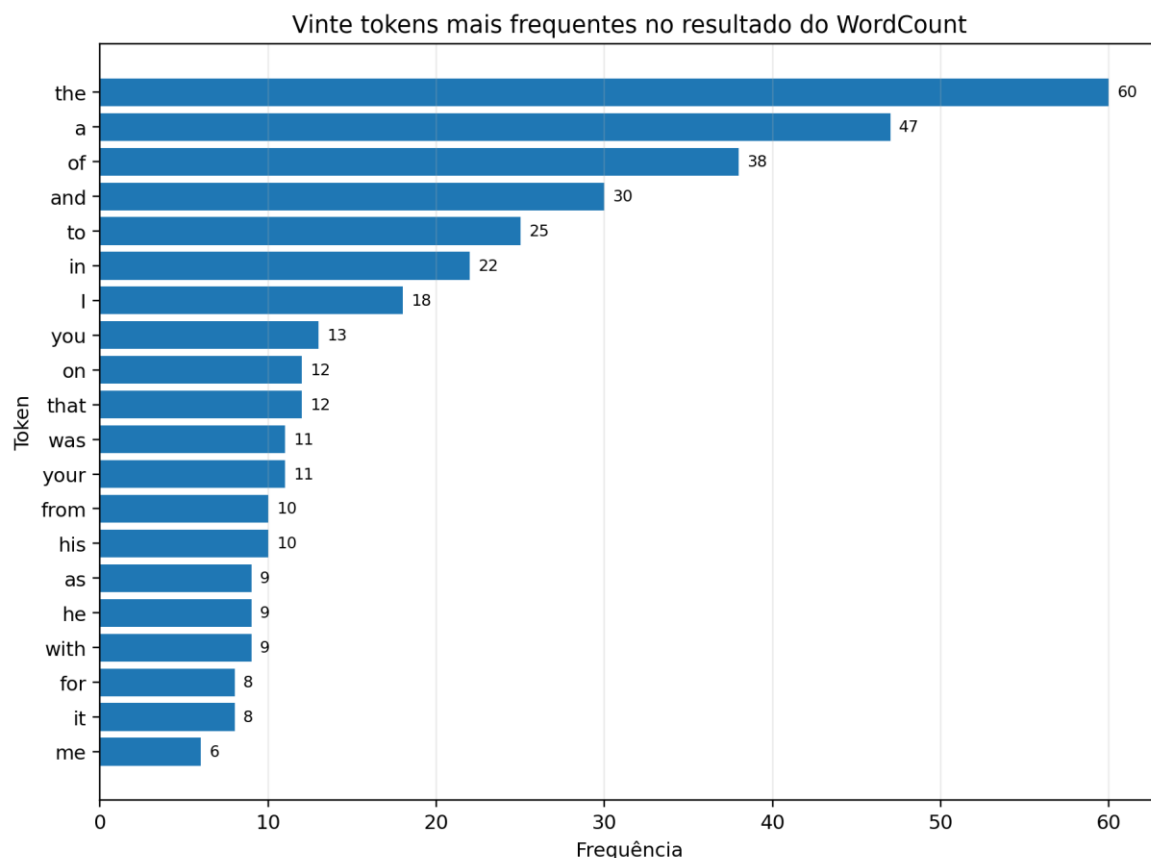


Figura 10-1: Distribuição das vinte maiores frequências

Os tokens mais frequentes foram “the” (60), “a” (47), “of” (38), “and” (30) e “to” (25). A saída evidencia que a tokenização foi realizada apenas por separação em espaços. Assim, formas com maiúsculas, minúsculas e sinais de pontuação foram consideradas distintas, como “The” e “the”, “Bulkington!” e “Bulkington?”, ou “Ipsum” e “ipsum”. Esse comportamento é coerente com a implementação baseada em *StringTokenizer* e deve ser considerado na interpretação estatística.

Como aprimoramento metodológico, uma versão posterior poderia normalizar os caracteres para minúsculas, remover pontuação, tratar apóstrofos e, conforme o objetivo analítico, excluir *stopwords*. Essas transformações reduziriam a fragmentação do vocabulário e produziriam frequências mais adequadas para análise textual, embora alterassem o caráter didático do *WordCount* original.

11 PROBLEMAS ENCONTRADOS E SOLUÇÕES ADOTADAS

A execução incluiu erros de natureza operacional e de interpretação do ambiente. A Tabela 6 sistematiza as ocorrências relevantes, suas causas e as medidas adotadas.

Tabela 11-1: Problemas, causas e soluções

Problema	Causa	Solução adotada
NoSuchBucket em tentativa anterior	Variável <i>BUCKET</i> apontava para um nome inexistente.	Obter o nome real por <i>Terraform</i> output ou <i>AWS S3 ls</i> e validar com <i>head-bucket</i> antes do sync.
404 ao copiar <i>S3://.../JARs/WordCount.JAR</i>	As reticências foram executadas literalmente no <i>WSL</i> antes da criação do <i>bucket/JAR</i> .	Não executar exemplos abreviados; usar o caminho real apenas depois do apply.
<i>Hadoop/HDFS</i> não encontrados no <i>WSL</i>	Os comandos pertenciam ao ambiente <i>EMR</i> , não à máquina local.	Mantê-los nos <i>Steps</i> do <i>EMR</i> ; não instalar <i>Hadoop</i> local para esta atividade.
<i>Step 3</i> originalmente executava <i>JAR</i> no <i>S3</i>	Uso incompatível do <i>Hadoop JAR</i> com URI remota no bash do <i>Step</i> .	Baixar o <i>JAR</i> para <i>/tmp/WordCount.JAR</i> antes da execução.
Cópia <i>HDFS</i> → <i>S3</i> potencialmente instável	Uso de <i>S3-dist-cp</i> na direção de saída.	Utilizar <i>copyToLocal</i> seguido de <i>AWS S3 cp --recursive</i> .
Correção reaplicada encontrou 0 trechos	O conteúdo original já havia sido substituído.	Interpretar como alteração já realizada e confirmar o <i>main.tf</i> com <i>grep</i> e <i>Terraform validate</i> .
FAILED_STEP_ID=None	Nenhum <i>Step</i> apresentou FAILED.	Não executar <i>describe-Step</i> quando o identificador retornado for None.
outputs.tf vazio	Os outputs estavam definidos no final de <i>main.tf</i> .	Utilizar <i>Terraform output</i> normalmente; não houve ausência funcional de outputs.

12 EVIDÊNCIAS, RASTREABILIDADE E REPRODUTIBILIDADE

As evidências foram organizadas no diretório `evidencias-finais-WordCount-20260717_125808`. A estrutura preserva tanto os registros sintéticos quanto os artefatos efetivamente utilizados e produzidos pelo processamento. A sincronização do *bucket* incluiu fontes, entrada, *JAR*, logs dos nós e dos *Steps*, `arquivo_SUCCESS` e resultado `part-r-000000`.

Tabela 12-1: Principais evidências preservadas

Arquivo/diretório	Conteúdo comprobatório
01-identidade-AWS.json	Identidade <i>AWS</i> utilizada na execução.
02-Terraform-outputs.txt	Identificadores do <i>cluster</i> , DNS, <i>bucket</i> e caminho de saída.
03-cluster-EMR.json	Descrição completa e estado do <i>cluster EMR</i> .
04-Steps-EMR.json	Estados e metadados dos quatro <i>Steps</i> .
05-conteudo-S3.txt	Listagem dos objetos armazenados no <i>bucket</i> .
S3/input/lorem.txt	Arquivo de entrada processado.
S3/JARs/WordCount.JAR	Aplicação compilada no <i>cluster</i> .
S3/output/_SUCCESS	Marcador de conclusão do job.
S3/output/part-r-00000	Resultado completo do <i>WordCount</i> .
S3/logs/...	Logs de provisionamento, nós e <i>Steps</i> do <i>EMR</i> .
results/part-r-00000	Cópia local simplificada do resultado.

A preservação das evidências permite verificar a correspondência entre os recursos declarados, os recursos efetivamente provisionados e os resultados produzidos. Para reexecução, é necessário renovar as credenciais do *Learner Lab*, garantir a disponibilidade das roles e da key pair, limpar qualquer estado anterior incompatível e aplicar o plano a partir da pasta *Terraform*.

13 DESCOMISSIONAMENTO E CONTROLE DE CUSTOS

No último registro analisado, o *cluster* `WordCount-EMR-cluster` estava em estado *WAITING*. Esse estado indica que os *Steps* terminaram, mas as instâncias permaneciam disponíveis. Portanto, o descomissionamento não deve ser considerado concluído até que haja evidência do *Terraform* destroy e da ausência de *clusters* ativos e do *bucket* criado pelo tutorial.

```
Terraform -chdir=Terraform destroy -auto-approve

AWS EMR list-clusters --cluster-states STARTING BOOTSTRAPPING RUNNING
WAITING --output table

AWS S3 ls | grep "650379606721-EMR-lab-WordCount" || echo "Bucket do
tutorial removido."
```

A execução do *destroy* deve ocorrer somente depois da confirmação das cópias locais. Como o *bucket* foi configurado com *force_destroy*, o *Terraform* pode remover seus objetos e o próprio *bucket*. A pasta local de evidências, armazenada no *OneDrive*, não é removida por esse procedimento.

14 CONCLUSÃO

A atividade atingiu o objetivo principal de executar um *WordCount MapReduce* em um ambiente *Hadoop* gerenciado na *AWS* e provisionado por infraestrutura como código. O processo validou a integração entre *Windows 11/WSL 2*, *AWS CLI*, *Terraform*, *Amazon S3*, *Amazon EMR*, *HDFS* e código *Java MapReduce*. Os quatro *Steps* foram concluídos, o *JAR* foi compilado no *cluster* e a saída foi persistida e baixada com sucesso.

A revisão prévia do *Terraform* foi determinante para a conclusão. As correções aplicadas ao acesso ao *JAR* e à movimentação da saída eliminaram dois pontos de falha recorrentes. Os erros observados durante a interação com o terminal foram solucionados sem comprometer a infraestrutura, e seu registro amplia o valor didático do trabalho ao distinguir claramente o ambiente local do ambiente remoto de execução.

Os resultados confirmam o funcionamento do processamento distribuído, mas também evidenciam uma limitação da tokenização por espaços: diferenças de caixa e pontuação geram tokens distintos. Como continuidade, recomenda-se experimentar uma etapa de normalização textual e comparar as frequências antes e depois do pré-processamento. Do ponto de vista operacional, permanece necessária a confirmação do descomissionamento dos recursos para encerrar adequadamente o ciclo de vida da infraestrutura e evitar consumo desnecessário do orçamento do *Learner Lab*.

15 APÊNDICE A – SEQUÊNCIA ESSENCIAL DE COMANDOS

A.1 Validação do ambiente e da identidade

```
AWS -version
Terraform version
Java -version
Javac -version
AWS sts get-caller-identity
```

A.2 Validação, planejamento e provisionamento

```
Terraform -chdir=Terraform fmt
Terraform -chdir=Terraform validate
Terraform -chdir=Terraform plan -out=tfplan
Terraform -chdir=Terraform apply tfplan
```

A.3 Recuperação de outputs e monitoramento

```
export CLUSTER_ID=$(Terraform -chdir=Terraform output -raw cluster_id)
export BUCKET=$(Terraform -chdir=Terraform output -raw S3_bucket)

AWS EMR list-Steps --cluster-id "$CLUSTER_ID" --query
'reverse(Steps)[].{Etapa:Name,Estado:Status.State}' --output table
```

A.4 Validação da saída, evidências e descomissionamento

```
AWS S3 ls "S3://$BUCKET/output/" -recursive
AWS S3 cp "S3://$BUCKET/output/part-r-00000" - | sort -t$' ' -
k2,2nr | head -20

AWS S3 sync "S3://$BUCKET/" "$EVID_DIR/S3/"
Terraform -chdir=Terraform destroy -auto-approve
```